

Paxos Made Simple

Paxos 通俗讲解

Leslie Lamport

2001 年 11 月 1 日

摘要

如果用普通的语言来表述，Paxos 算法其实非常简单。

目录

1	引言	1
2	共识算法	1
2.1	问题	1
2.2	选定一个值	2
2.3	获知被选定的值	9
2.4	进展	10
2.5	实现	10
3	实现状态机	11

1 引言

用于实现容错分布式系统的 Paxos 算法一直被认为很难理解，这是因为它最初的表述对许多读者来说像希腊文一样晦涩 [5]。事实上，它是最简单、最显然的分布式算法之一。它的核心是一个共识算法，也就是 [5] 中的“synod”（教会）算法。下一节将说明，这个共识算法几乎不可避免地从我希望它满足的性质中推导出来（译者注：这句话有点拗口，它其实表述的意思是，我们可以自然地假设我们希望这个算法拥有某种“特性”，然后从这些特性分析，自然而然地导出 Paxos 算法流程）。最后一节会解释完整的 Paxos 算法：只要把共识算法直接应用到构建分布式系统的状态机方法上，就能得到它。状态机的做法应该已经广为人知，因为它是分布式系统理论中也许被引用最多的那篇文章 [4] 的主题。

2 共识算法

2.1 问题

假设有一组进程可以提出值。共识算法要保证：在这些被提出的值中，只会有一个值被选定。如果没有任何值被提出，那么就不应该有值被选定。如果某个值已经被选定，那么进程应该能够得知这个被选定的值。共识的安全性要求如下：

- 只有被提出过的值才可以被选定；
- 只有一个值会被选定；
- 进程只有在某个值确实已经被选定时，才会得知该值已被选定。

我们不试图精确规定活性要求（译者注：这意味着我们只需要保证正确性，最终能让它达成一致就行了，对于效率或者后续提及的“活锁”导致很难才能选出值的问题不关心，工业界会有成熟的方法解决这个问题）。不

过，目标是保证某个被提出的值最终会被选定；并且如果某个值已经被选定，那么进程最终能够得知这个值。

我们让三类 agent 承担共识算法中的三种角色：proposer、acceptor 和 learner。在一种实现中，单个进程可以扮演多个 agent，但 agent 到进程的映射关系不是这里关心的问题。

假设 agent 之间可以通过发送消息来通信。我们采用通常的异步、非拜占庭模型，其中：

- Agent 可以以任意速度运行，可以因停止而失败，也可以重启。由于所有 agent 都可能在某个值被选定后失败并重启，所以除非失败后重启的 agent 能够记住某些信息 (译者注：这里暗示了使用非易失存储，例如磁盘)，否则不可能得到一个解。
- 消息可能经过任意长时间才送达，可能被复制，也可能丢失，但不会被篡改 (译者注：所有节点都是“诚实”的，不会伪造消息)。

2.2 选定一个值

选定一个值最简单的办法是只有一个 acceptor agent。Proposer 向 acceptor 发送一个 proposal，而 acceptor 选择它收到的第一个被提出的值。这个方案虽然简单，但并不令人满意，因为一旦这个 acceptor 失败，系统就无法继续取得进展。(译者注：我们希望系统整体是一定程度上可容错的，部分节点失败不影响整体系统的运行；但是超过一定数量的节点失败，系统就无法继续运行，也是可以接受的。我们不希望单个节点失败不应该影响系统的运行)

所以，我们试试另一种选定值的办法：使用多个 acceptor。Proposer 把一个被提出的值发送给一组 acceptor。Acceptor 可以接受这个被提出的值。当足够大的一组 acceptor 已经接受它时，这个值就被选定了。多大才算足够大？为了保证只有一个值会被选定，我们可以让“足够大的一组”指任意多数派 acceptor (译者注：在本论文，acceptor 数量是不会改变的，文章不

讨论成员变更的情况，所以多数派的定义是固定的。例如，在数量为 3 个的情况下，多数派就是 2 个或者 3 个。在数量为 5 个的情况下，多数派就是 3/4/5 个)。因为任意两个多数派至少有一个 acceptor 相交，所以只要每个 acceptor 至多接受一个值，这个办法就能成立。(对“多数派”概念存在一种显然的推广，这种推广已在大量论文中被讨论，显然最早可追溯到文献 [3]。)

在没有失败和消息丢失的情况下，即使只有一个 proposer 提出一个值，我们也希望有值被选定 (译者注：see? 这就是我之前提到的，论文整体的行文思路是“通过提出一些我们需要的一些特性来推倒算法流程”，这显然是我们希望满足的第一个特性)。这给出了下面的要求：

P1. Acceptor 必须接受它收到的第一个 proposal。

但这个要求带来了一个问题。多个不同的 proposer 可能几乎同时提出多个值，导致每个 acceptor 都已经接受了一个值，但没有任何一个值被多数 acceptor 接受。即使只有两个被提出的值，如果每个值都被大约一半 acceptor 接受，那么只要一个 acceptor 失败，就可能无法知道到底哪个值被选定。(译者注：这里其实作者有两个假设，一个是 P1——也就是 acceptor 必须接受它收到的第一个 proposal；第二个假设是 acceptor 只能接受一个 proposal。举例来说，假如有 4 个 acceptor，两个 proposer 提出了两个不同的值，导致每个 value 都被不同的两个 acceptor 接受，那么没有一个 value 被多数派接受。这时，值就不能被选出来，直接流程死锁)

P1 以及“只有当一个值被多数 acceptor 接受时才被选定”这个要求意味着：必须允许 acceptor 接受不止一个 proposal (译者注：无奈之举，acceptor 只能接受一个 proposal 必然不行，那么只能允许接受多个了)。我们给每个 proposal 分配一个自然数编号，用来区分 acceptor 可能接受的不同 proposal。因此，一个 proposal 由 proposal 编号和值组成 (译者注：后续把它们称为 *proposal number* 和 *proposal value*)。为了避免混淆，我们要求不同 proposal (译者注：什么叫不同的 proposal? 这里很微妙，这里可以暂且理解为“全局/全世界维度”只会有一份 (*proposal number, proposal value*) tuple

对,且不会存在两个 *proposal number* 相同的 *proposal*。在这种全局 *proposal number* 唯一的情况下,一个 *proposal number* 只对应一个 *value*, *proposal number* 就是 *proposal* 的“唯一身份证”)具有不同的 *proposal number*。如何做到这一点(译者注:这里指的是如何做到“全局/全世界”维度的 *proposal*,毕竟不同的 *agent* 有可能提出一样的 *proposal number*,这样就撞号了)取决于具体实现,所以现在我们先直接假设它成立。当某个带有值 v 的 *proposal* 被多数 *acceptor* *accept*(接受)时,这个值就被“选定”。在这种情况下,我们说这个 *proposal*(以及它的值)已经被 *chosen*。(译者注:我举个例子,假如一个 *proposal* 是 $(5,3)$,它被 *chosen*,则代表 *proposal number*=5 的 *proposal* 被选定且 *value*=3 被 *chosen*)

我们可以允许多个 *proposal* 被 *chosen*,但必须保证所有被选定的 *proposal* 都有相同的值。按 *proposal* 编号归纳,只要保证下面这一点就足够:

P2. 如果一个值为 v 的 *proposal* 被选定,那么每个编号更大的、被选定的 *proposal* 都具有值 v 。(译者注:这里很好理解,假如 *agent 1* 提出 *proposal*(5,3),且被全部 *acceptor* *accept* 了,那么事实上该 *proposal* 就被 *chosen* 了;随后 *agent 2* 提出 *proposal*(6,4),且也被全部 *acceptor* *accept* 了,那么其实就“覆写了”之前的工作。因此为了避免这样的事情发生,就提出了 P2 这个要求)

由于 *proposal number* 是全序(译者注:意味着任何两个 *proposal number* 之间都有明确的大小关系,且由于之前作者提到了 *proposal number* 是“全局唯一”的假设(虽然作出了假设,但是没说具体怎么实现的,后续后说的),任何两个 *proposal* 都不会相等)的,条件 P2 保证了最关键的安全性性质:只有一个值会被 *chosen*。

一个 *proposal* 要被 *chosen*,至少必须被一个 *acceptor* *accept*。所以,我们可以通过满足下面这个条件来满足 P2:

P2a. 如果一个值为 v 的 *proposal* 被 *chosen*,那么任何 *acceptor* *accept* 的每个编号更大的 *proposal* 都具有值 v 。

我们仍然保持 P1，以保证会有某个 proposal 被 chosen。由于通信是异步的，某个 proposal 可能已经被 chosen，而某个特定的 acceptor c 从未收到过任何 proposal。假设一个新的 proposer “醒来”，并发出一个编号更大、但值不同的 proposal。P1 要求 c 接受这个 proposal，从而违反 P2a。要同时保持 P1 和 P2a，就必须把 P2a 加强为：

P2b. 如果一个值为 v 的 proposal 被 chosen，那么任何 proposer 发出的每个编号更大的 proposal 都具有值 v 。(译者注：这里 P2a 是对 acceptor 作出了要求，要求 accept 满足条件。然而 acceptor 其实在 Paxos 的设计上更倾向于“孤立”的，不感知其它 acceptor 和全局状态，所以这里从源头入手，扩展成了 P2b，也就是对 proposer 作出要求)

因为 proposal 必须先由 proposer 发出，才可能被 acceptor 接受，所以 P2b 蕴含 P2a，而 P2a 又蕴含 P2。(译者注：蕴含关系就是包含关系，P2b 能推出 P2a，P2a 能推出 P2，也就是说只要满足 P2b，之前提到的 P2a 和 P2 也就满足了)

为了弄清楚如何满足 P2b，我们先考虑怎样证明它成立 (译者注：这里的前提是 (m, n) 已经被 chosen，问题是怎样设计协议，才能让这个条件一直成立)。我们会假设某个编号为 m 、值为 v 的 proposal 已经被 chosen，然后证明任何编号为 $n > m$ 的 proposal 也具有值 v 。为了让证明更容易，我们可以对 n 做归纳；也就是说，在额外假设所有编号位于 $m..(n-1)$ 之间的已发出 proposal 都具有值 v 的前提下，证明编号为 n 的 proposal 具有值 v ，其中 $i..j$ 表示从 i 到 j 的所有编号集合。编号为 m 的 proposal 要被 chosen，必然存在某个由多数 acceptor 组成的集合 C ，使得 C 中的每个 acceptor 都 accept 了它。把这一点与归纳假设结合起来，“ m 已被 chosen”这个假设意味着：

C 中的每个 acceptor 都已经 accept 了某个编号位于 $m..(n-1)$ 之间的 proposal，并且任何 acceptor accept 的、编号位于 $m..(n-1)$ 之间的每个 proposal 都具有值 v 。

由于任何由多数 acceptor 组成的集合 S 都至少包含 C 中的一个成员, 所以只要确保下面这个不变式一直成立 (译者注: 这里导出 P2c 算这个论文最天马行空的地方了, 它论述的是在 (m,n) 已经被 *chosen* 的前提下, 怎样加强协议, 让协议更可“操作化”的同时保证 P2b), 我们就能推出编号为 n 的 proposal 具有值 v :

P2c. 对任意 v 和 n , 如果一个值为 v 、编号为 n 的 proposal 被发出, 那么存在一个由多数 acceptor 组成的集合 S , 满足以下二者之一: (a) S 中没有 acceptor accept 过任何编号小于 n 的 proposal; 或者 (b) 在 S 中所有 acceptor accept 过的、编号小于 n 的 proposal 里, 编号最高的那个 proposal 的值是 v 。

因此, 只要保持 P2c 这个不变式, 我们就可以满足 P2b。

为了保持 P2c 这个不变式, 想要发出编号为 n 的 proposal 的 proposer 必须获知: 在某个多数 acceptor 集合中的每个 acceptor 那里, 编号小于 n 且已经被 accept 或将会被 accept 的 proposal 里, 编号最高的那个是什么 (如果存在的话)。了解已经被 accept 的 proposal 很容易; 预测未来会 accept 什么则很困难。Proposer 不去预测未来, 而是通过取得承诺来控制未来: 让这些 acceptor 承诺不会再 accept 这样的 proposal。换句话说, proposer 请求 acceptor 不要再 accept 任何编号小于 n 的 proposal。这导出了如下发出 proposal 的算法。(译者注: 询问所有节点是不明智的, 因为任何机器都会以随机的速度运行, 状态随时会因为其它的 *proposal* 更改, 在没有全局锁的情况下, 只能放弃预测未来, 而是通过取得承诺来控制未来)

1. Proposer 选择一个新的 proposal 编号 n , 向某个 acceptor 集合的每个成员发送请求, 要求它回复:
 - (a) 一个承诺: 以后绝不再接受编号小于 n 的 proposal;
 - (b) 它已经 accept 过的、编号小于 n 的最高编号 proposal (如果存在的话)。

我把这样的请求称为编号为 n 的 prepare 请求。

2. 如果 proposer 收到了多数 acceptor 给出的所需响应 (译者注: *proposer* 向所有 *acceptor* 发送 *prepare(n)*, 只要多数派 say ok, 那么就能进入下一步; 如果拿不到多个 *acceptor* 的响应只能选择重试), 那么它就可以发出编号为 n 、值为 v 的 proposal (译者注: 也就是接下来会提到的 *accept* 请求)。这里的 v 是这些响应中最高编号 proposal 的值; 如果 acceptors 都报告没有 accept 过 proposal (译者注: 这个报告来自于 *prepare* 请求的返回), 那么 v 可以是 proposer 自己选择的任意值 (译者注: 这里常见的做法是提出客户端传入的值; 这里还有一个微妙的点, 它强调了“都没有” (这里值的是返回的多个响应中都没有, 不一定要大家全部都响应) *accept* 过 *proposal* 才能选择自己的值, 否则就只能选择拿到的响应中最高 *proposal number* 对应的值了。))。

Proposer 通过向某个 acceptor 集合发送请求, 请求它们 accept 这个 proposal, 来发出 proposal。 (这个集合不必与响应初始请求的 acceptor 集合相同 (译者注: 事实上最好是向所有的 *acceptors* 发送请求)。) 我们把这种请求称为 accept 请求。

上面描述的是 proposer 的算法。那么 acceptor 呢? Acceptor 可能从 proposer 收到两类请求: prepare 请求和 accept 请求。Acceptor 可以忽略任何请求而不破坏安全性 (译者注: 分布式算法的特征就是能容忍任何丢包, *dup*, 以及乱序, 及时在极端的丢包场景下只要 *agent* 不断重试就理论能成功)。所以, 我们只需要说明它何时允许响应某个请求。它总是可以响应 prepare 请求。它可以响应 accept 请求并 accept 该 proposal, 当且仅当它没有承诺不这样做。换句话说:

P1a. Acceptor 可以 accept 编号为 n 的 proposal, 当且仅当它尚未响应过任何编号大于 n 的 prepare 请求。

注意, P1a 包含了 P1。

现在我们已经有了一个完整的、满足所需安全性性质的选值算法，前提是 proposal 编号唯一。最终算法还要加入一个小优化。

假设某个 acceptor 收到编号为 n 的 prepare 请求，但它已经响应过编号大于 n 的 prepare 请求，因此已经承诺不会 accept 任何新的编号为 n 的 proposal。此时这个 acceptor 没有理由响应新的 prepare 请求，因为它不会 accept 该 proposer 想要发出的编号为 n 的 proposal。所以，我们让 acceptor 忽略这样的 prepare 请求。我们也让它忽略针对它已经 accept 过的 proposal 的 prepare 请求。

有了这个优化，acceptor 只需要记住它曾经 accept 过的最高编号 proposal，以及它已经响应过的最高编号 prepare 请求的编号。由于无论是否发生失败，P2c 都必须保持为不变式，所以即使 acceptor 失败后重启，也必须记住这些信息。注意，proposer 总是可以放弃一个 proposal，并完全忘记它，只要它绝不再尝试发出另一个具有相同编号的 proposal 即可。

把 proposer 和 acceptor 的动作合在一起，可以看到算法分两阶段运行。

阶段 1。(a) Proposer 选择一个 proposal 编号 n ，向多数 acceptor 发送编号为 n 的 prepare 请求。

(b) 如果 acceptor 收到编号为 n 的 prepare 请求，且 n 大于它已经响应过的任何 prepare 请求编号，那么它响应这个请求：承诺不再 accept 任何编号小于 n 的 proposal，并附上它已经接受过的最高编号 proposal(如果存在的话)。

阶段 2。(a) 如果 proposer 收到多数 acceptor 对其编号为 n 的 prepare 请求的响应，那么它向这些 acceptor 中的每一个发送 accept 请求，请求它们接受编号为 n 、值为 v 的 proposal。这里的 v 是所有响应中最高编号 proposal 的值；如果响应报告没有 proposal，那么 v 可以是任意值。

(b) 如果 acceptor 收到针对编号为 n 的 proposal 的 accept 请求，除非它已经响应过编号大于 n 的 prepare 请求，否则它接受该 proposal。

只要对每个 proposal 都遵循这个算法，一个 proposer 可以发出多个 proposal。它也可以在协议中途的任何时刻放弃某个 proposal。(即使该 proposal 的请求和/或响应在它被放弃很久之后才到达目的地，正确性仍然保

持。) 如果某个 proposer 已经开始尝试发出编号更大的 proposal, 那么放弃当前 proposal 很可能是个好主意。因此, 如果 acceptor 因为已经收到过编号更高的 prepare 请求而忽略某个 prepare 或 accept 请求, 那么它大概应该告知 proposer; proposer 随后应该放弃自己的 proposal。这是一种不影响正确性的性能优化。

2.3 获知被选定的值

为了得知某个值已经被选定, learner 必须知道某个 proposal 已经被多数 acceptor 接受。显然的算法是: 每当 acceptor 接受某个 proposal 时, 就向所有 learner 回复并发送这个 proposal。这样 learner 可以尽快知道被选定的值, 但它要求每个 acceptor 都向每个 learner 回复, 响应数量等于 acceptor 数量与 learner 数量的乘积。

非拜占庭失败这个假设使得一个 learner 很容易从另一个 learner 那里得知某个值已经被接受。我们可以让 acceptor 把自己的 acceptance 发送给一个指定 learner, 再由这个指定 learner 在某个值被选定时通知其他 learner。这种做法需要额外一轮通信, 所有 learner 才能发现被选定的值。它的可靠性也较差, 因为指定 learner 可能失败。但它需要的响应数量只等于 acceptor 数量和 learner 数量之和。

更一般地, acceptor 可以把自己的 acceptance 发送给某一组指定 learner, 其中每个指定 learner 都可以在某个值被选定时通知所有 learner。使用更大的指定 learner 集合可以提高可靠性, 代价是更高的通信复杂度。

由于消息可能丢失, 一个值可能已经被选定, 却没有任何 learner 知道。Learner 可以询问 acceptor 它们接受过哪些 proposal, 但如果某个 acceptor 失败, 就可能无法知道某个特定 proposal 是否已经被多数 acceptor 接受。在这种情况下, learner 只有当一个新的 proposal 被选定时, 才会知道被选定的值是什么。如果某个 learner 需要知道是否已经有值被选定, 它可以让一个 proposer 使用前面描述的算法发出一个 proposal。

2.4 进展

很容易构造这样一种场景：两个 proposer 各自不断发出一系列编号递增的 proposal，但没有任何 proposal 被选定。Proposer p 完成了编号为 n_1 的 proposal 的阶段 1。另一个 proposer q 随后完成了编号为 $n_2 > n_1$ 的 proposal 的阶段 1。由于 acceptor 都已经承诺不再接受任何编号小于 n_2 的新 proposal，所以 proposer p 针对编号为 n_1 的 proposal 发出的阶段 2 accept 请求会被忽略。于是 proposer p 又开始并完成编号为 $n_3 > n_2$ 的新 proposal 的阶段 1，导致 proposer q 第二阶段的 accept 请求被忽略。如此往复。

为了保证进展，必须选择一个指定 proposer，只有它尝试发出 proposal。如果指定 proposer 能够与多数 acceptor 成功通信，并且使用的 proposal 编号大于所有已使用编号，那么它将成功发出一个会被接受的 proposal。如果它得知某个请求带有更高 proposal 编号，就放弃自己的 proposal 并重试，那么这个指定 proposer 最终会选择一个足够高的 proposal 编号。

因此，如果系统中足够多的部分 (proposer、acceptor 和通信网络) 正常工作，就可以通过选出单个指定 proposer 来实现活性。Fischer、Lynch 和 Patterson 的著名结果 [1] 说明，用来选举 proposer 的可靠算法必须使用随机性或真实时间，例如使用超时。不过，无论选举成功还是失败，安全性都能得到保证。

2.5 实现

Paxos 算法 [5] 假设存在一个由进程组成的网络。在它的共识算法中，每个进程都扮演 proposer、acceptor 和 learner 的角色。算法会选择一个 leader，由它扮演指定 proposer 和指定 learner。Paxos 共识算法正是上面描述的算法，请求和响应作为普通消息发送。(响应消息会带上对应的 proposal 编号，以免混淆。) 失败期间仍能保存的稳定存储用来维护 acceptor 必须记住的信息。Acceptor 在实际发送响应之前，会先把它打算给出的响应记录到稳定存储中。

剩下要描述的只有一种机制：如何保证永远不会有两个 proposal 以相同编号被发出。不同 proposer 从彼此不相交的编号集合中选择编号，所以两个不同的 proposer 绝不会发出编号相同的 proposal。每个 proposer 会在稳定存储中记住它曾经尝试发出的最高编号 proposal，并在阶段 1 中使用一个高于自己已经用过的所有编号的 proposal 编号。

3 实现状态机

实现分布式系统的一种简单方法是：让一组客户端向一个中央服务器发出命令。这个服务器可以被描述为一个确定性状态机，它按照某个顺序执行客户端命令。状态机具有当前状态；它通过接收一个命令作为输入，并产生一个输出和一个新状态来执行一步。例如，一个分布式银行系统的客户端可能是柜员，而状态机状态可能包含所有用户的账户余额。取款操作可以通过执行一条状态机命令来完成：当且仅当账户余额大于取款金额时减少该账户余额，并把旧余额和新余额作为输出。

使用单个中央服务器的实现会在该服务器失败时失效。因此，我们改为使用一组服务器，每个服务器都独立实现该状态机。由于状态机是确定性的，如果所有服务器都执行同一串命令，它们就会产生相同的状态序列和输出序列。发出命令的客户端随后可以使用任意服务器为它生成的输出。

为了保证所有服务器都执行同一串状态机命令，我们实现一系列彼此独立的 Paxos 共识算法实例，其中第 i 个实例选定的值就是命令序列中的第 i 条状态机命令。每个服务器在算法的每个实例中都扮演所有角色 (proposer、acceptor 和 learner)。现在，我先假设服务器集合是固定的，因此所有共识算法实例都使用同一组 agent 集合。

在正常运行中，系统会选出单个服务器作为 leader。这个 leader 在所有共识算法实例中充当指定 proposer(唯一尝试发出 proposal 的 proposer)。客户端把命令发送给 leader，由 leader 决定每条命令应该出现在序列中的哪个位置。如果 leader 决定某条客户端命令应该成为第 135 条命令，它就尝试让这条命令成为第 135 个共识算法实例选定的值。通常它会成功。它

可能因为失败而失败，也可能因为另一个服务器也认为自己是 leader，并且对第 135 条命令应该是什么有不同想法而失败。但共识算法保证，至多只有一条命令会被选为第 135 条。

这种方法高效的关键在于，在 Paxos 共识算法中，要提出的值直到阶段 2 才被选择。回想一下，在完成 proposer 算法的阶段 1 之后，要么要提出的值已经被确定，要么 proposer 可以自由提出任意值。

现在我将描述 Paxos 状态机实现在正常运行期间如何工作。之后再讨论可能出错的情况。我考虑这样一种情形：前一个 leader 刚刚失败，新的 leader 已经被选出。（系统启动是一个特例，此时还没有任何命令被提出。）

新的 leader 作为所有共识算法实例中的 learner，应该知道大多数已经被选定的命令。假设它知道命令 1-134、138 和 139，也就是共识算法实例 1-134、138 和 139 中被选定的值。（后面会看到命令序列中这样的空洞是怎样产生的。）然后它执行实例 135-137 以及所有大于 139 的实例的阶段 1。（下面会说明这是怎么做的。）假设这些执行结果决定了实例 135 和 140 中要提出的值，但其他所有实例中的待提议值都没有受到约束。Leader 随后对实例 135 和 140 执行阶段 2，从而选定命令 135 和 140。

此时，leader 以及任何获知了 leader 所知全部命令的其他服务器，都可以执行命令 1-135。不过，它还不能执行同样已经知道的命令 138-140，因为命令 136 和 137 尚未被选定。Leader 可以把客户端接下来请求的两条命令作为命令 136 和 137。相反，我们让它立即用一种特殊的“no-op”命令填补空洞；这种命令作为命令 136 和 137 被提出，并且不会改变状态。（它通过执行共识算法实例 136 和 137 的阶段 2 来做到这一点。）一旦这些 no-op 命令被选定，命令 138-140 就可以执行了。

现在，命令 1-140 都被选定。Leader 也已经完成了所有大于 140 的共识算法实例的阶段 1，并且可以在这些实例的阶段 2 中自由提出任意值。它把编号 141 分配给客户端请求的下一条命令，并在共识算法实例 141 的阶段 2 中把它作为值提出。它把收到的下一条客户端命令作为命令 142 提出，依此类推。

Leader 可以在得知它提出的命令 141 已经被选定之前，就提出命令

142。它在提出命令 141 时发送的所有消息都可能丢失，而命令 142 可能
在其他任何服务器知道 leader 把什么作为命令 141 提出之前就被选定。当
leader 没有收到实例 141 中阶段 2 消息的预期响应时，它会重传这些消息。
如果一切顺利，它提出的命令会被选定。不过，它也可能先失败，在已选定
命令序列中留下一个空洞。一般来说，假设 leader 可以领先 α 条命令，也
就是说，在命令 1 到 i 已经被选定之后，它可以提出命令 $i+1$ 到 $i+\alpha$ 。那
么就可能产生最多 $\alpha-1$ 条命令的空洞。

新选出的 leader 会为无穷多个共识算法实例执行阶段 1。在上面的场
景中，就是实例 135–137 以及所有大于 139 的实例。通过对所有实例使用
相同的 proposal 编号，它可以用一条相当短的消息把这件事发送给其他服
务器。在阶段 1 中，acceptor 只有在已经从某个 proposer 那里收到过阶段
2 消息时，才会回复比简单 OK 更多的信息。（在这个场景中，只有实例 135
和 140 属于这种情况。）因此，作为 acceptor 的服务器可以用一条相当短
的消息回复所有实例。所以，执行这无穷多个阶段 1 实例并不会造成问题。

由于 leader 失败和新 leader 选举应该是少见事件，执行一条状态机命
令的有效成本，也就是对该命令/值达成共识的有效成本，就是只执行共识
算法阶段 2 的成本。可以证明，在存在故障的情况下，Paxos 共识算法的阶
段 2 具有所有达成一致算法中可能的最低成本 [2]。因此，Paxos 算法本质
上是最优的。

上述关于系统正常运行的讨论假设系统总是只有一个 leader，除了当
前 leader 失败和新 leader 选举之间的一小段时间。在异常情况下，leader
选举可能失败。如果没有服务器作为 leader，那么就不会有新命令被提出。
如果多个服务器都认为自己是 leader，那么它们可能在同一个共识算法实
例中都提出值，这可能阻止任何值被选定。不过，安全性仍然保持：两个不
同服务器绝不会对第 i 条状态机命令选定的值产生分歧。选出单个 leader
只是为了保证进展。

如果服务器集合可以变化，那么必须有办法确定哪些服务器实现哪些
共识算法实例。最简单的办法是通过状态机本身来完成。当前服务器集合
可以成为状态的一部分，并且可以通过普通状态机命令来改变。我们可以

允许 leader 领先 α 条命令，方式是让执行共识算法实例 $i + \alpha$ 的服务器集合由第 i 条状态机命令执行之后的状态来指定。这允许我们简单地实现任意复杂的重配置算法。

参考文献

- [1] Michael J. Fischer, Nancy Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, April 1985.
- [2] Idit Keidar and Sergio Rajsbaum. On the cost of fault-tolerant consensus when there are no faults—a tutorial. Technical Report MIT-LCS-TR-821, Laboratory for Computer Science, Massachusetts Institute Technology, Cambridge, MA, 02139, May 2001. Also published in *SIGACT News* 32(2), June 2001.
- [3] Leslie Lamport. The implementation of reliable distributed multiprocess systems. *Computer Networks*, 2:95–114, 1978.
- [4] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978.
- [5] Leslie Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, May 1998.